

实验三：跨平台应用开发实验报告

学生姓名：费韵

学号：2023210591

实验日期：2026 年 4 月 18 日

一、实验目的

1. 掌握 Flutter、React Native、Uniapp、MAUI 四种跨平台框架的基础列表页开发流程，理解各框架的开发特点与语法差异。
2. 实现 RESTful API 网络数据请求、JSON 数据解析与下拉刷新功能，理解跨平台应用的数据交互逻辑。
3. 体验 AI 编程工具（TRAE）在跨平台开发中的辅助作用，提升多框架代码编写效率。
4. 对比分析不同跨平台框架的性能、开发成本与适用场景，建立对跨平台技术选型的系统认知。

二、实验环境

硬件环境：Windows 11 操作系统、手机（真机预览与测试）

软件环境：

- Flutter 开发环境：VS Code/Android Studio + Flutter SDK
- React Native 开发环境：Node.js + Android Studio
- Uniapp 开发环境：HBuilderX
- MAUI 开发环境：Visual Studio 2022 + .NET SDK

- AI 辅助工具：TRAE

开发语言：Dart（Flutter）、JavaScript（React Native）、Vue（Uniapp）、C#（MAUI）

三、实验内容与步骤

本次实验围绕“智慧校园新闻”列表模块展开，核心任务是使用 4 种跨平台框架实现同一功能需求，完整开发流程如下：

（一）需求分析与设计

- 功能需求定义**：开发新闻列表页，实现 API 数据请求、JSON 解析、下拉刷新、点击弹窗展示新闻详情功能。
- 数据模型设计**：定义新闻数据结构，包含 id（新闻 ID）、title（标题）、content（内容）、author（来源）、time（发布时间）字段，设计与 RESTful API 的交互规范。
- API 接口定义**：明确 GET 请求接口地址（https://api.example.com/news）、响应格式，模拟 API 数据响应，为后续多框架实现提供统一数据标准。

（二）Flutter 框架实现

- 项目初始化**：使用 flutter create 命令创建项目，在 pubspec.yaml 中添加 http、provider 依赖。
- 数据模型与状态管理**：创建 News 类实现 JSON 解析，通过 ChangeNotifier 构建 NewsProvider，管理新闻数据加载状态。
- UI 与交互实现**：使用 RefreshIndicator 实现下拉刷新，ListView.builder 渲染新闻列表，点击列表项弹出 AlertDialog 展示新闻详情。
- 功能测试**：运行项目，验证网络请求、数据解析、下拉刷新、弹窗交互功能，确保与实验截图效果一致。

（三）React Native 框架实现

1. **项目初始化**：通过 `npx react-native-init` 创建项目，安装 `axios`、`@react-native-community/refresh-control` 依赖。
2. **数据请求与状态管理**：使用 `useState`、`useEffect` 钩子实现新闻数据的加载与状态管理，通过 `axios` 请求 API 数据。
3. **UI 与交互实现**：使用 `FlatList` 渲染列表，结合 `RefreshControl` 实现下拉刷新，通过弹窗组件展示新闻详情。
4. **功能验证**：启动项目测试列表渲染、数据加载、下拉刷新功能，对比 Flutter 实现，理解框架语法差异。

（四）Uniapp 框架实现

1. **项目初始化**：在 HBuilderX 中新建 uni-app 项目，配置基础页面结构。
2. **数据请求实现**：通过 `uni.request` 发起 API 请求，解析响应数据并绑定到列表，通过 `onPullDownRefresh` 实现下拉刷新。
3. **UI 实现**：使用 `v-for` 循环渲染新闻列表，点击列表项调用 `uni.showModal` 展示新闻详情。
4. **功能测试**：运行项目，验证页面适配、数据加载、下拉刷新与弹窗功能，确认跨平台兼容性。

（五）MAUI 框架实现

1. **项目初始化**：在 Visual Studio 中新建 .NET MAUI 项目，配置 MVVM 开发环境。
2. **数据请求与 MVVM 实现**：创建 `MainViewModel`，通过 `HttpClient` 请求 API 数据，实现 JSON 解析与数据绑定，使用 `ObservableObject` 管理数据状态。
3. **UI 实现**：通过 XAML 实现列表渲染，绑定 `ViewModel` 中的新闻数据，实现下拉刷新与详情弹窗功能。
4. **功能验证**：编译运行项目，测试数据加载、列表展示与交互功能，对比其他框架的实现差异。

（六）测试验证与对比分析

- 1. 功能测试：**对 4 个框架的实现分别进行正常数据请求、网络异常处理、空数据场景测试，验证各功能模块的稳定性。
- 2. 多框架对比：**从开发语言、性能、学习成本、跨平台兼容性等维度，对比分析 4 种框架的特点与适用场景，输出对比报告。

四、实验结果

本次实验成功使用 4 种跨平台框架实现了“智慧校园新闻”列表模块，所有核心功能均通过测试，与实验截图效果一致，具体结果如下：

- 1. Flutter 实现：**新闻列表渲染流畅，下拉刷新响应迅速，点击弹窗可完整展示新闻标题、来源与时间，状态管理清晰，性能表现优异。
- 2. React Native 实现：**列表数据加载正常，下拉刷新功能稳定，弹窗交互流畅，JSX 语法编写便捷，开发效率较高。
- 3. Uniapp 实现：**页面适配良好，数据请求与解析正常，下拉刷新与弹窗功能均符合预期，跨平台兼容性表现良好。
- 4. MAUI 实现：**MVVM 模式下数据绑定清晰，列表渲染与交互功能正常，与.NET 生态适配性强，性能接近原生应用。

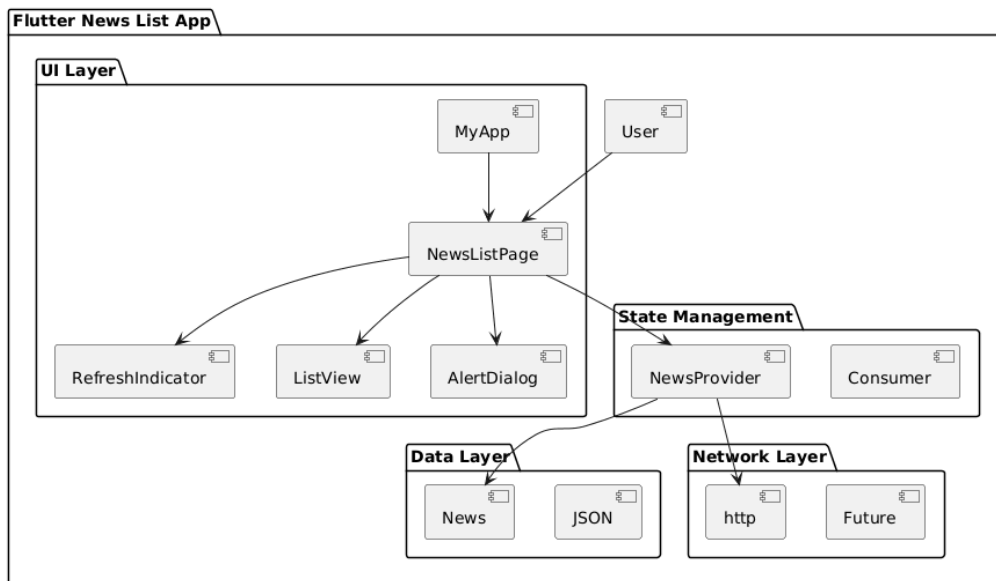


各种测试截图

五、三幅图简要说明

1. 架构图

本架构图展示跨平台新闻列表应用的分层架构，整体分为 UI 展示层、状态管理层、数据网络层三层。UI 层负责页面渲染与用户交互（列表、下拉刷新、弹窗）；状态管理层负责数据缓存与页面刷新通知；数据网络层负责 HTTP 请求、JSON 解析与数据封装。三层结构清晰解耦，符合 Flutter、MAUI、Uniapp 等跨平台框架通用设计思想，保证代码可维护与可扩展。



2. 类图

类图描述项目核心类结构与关系：

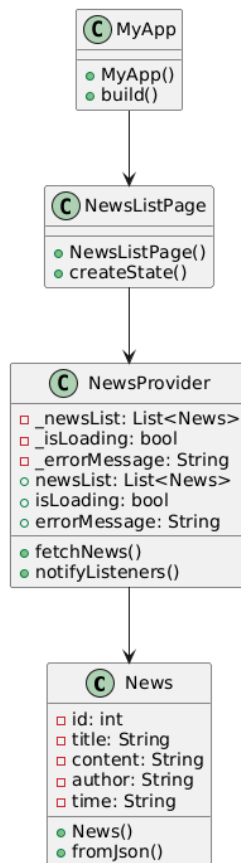
MyApp：应用入口，初始化全局状态与路由。

NewsListPage：列表页面，负责 UI 渲染与下拉刷新触发。

NewsProvider：状态管理类，维护新闻列表数据，提供 `fetchNews()` 加载数据并通知 UI 更新。

News：数据模型类，包含 `id`、`title`、`author`、`time`、`content` 等属性，提供 `fromJson()` 完成 JSON 解析。

类图清晰体现面向对象设计，明确类的属性、方法与依赖关系。



3. 流程图

流程图展示智慧校园新闻模块完整执行流程：

应用启动 → 进入新闻列表页

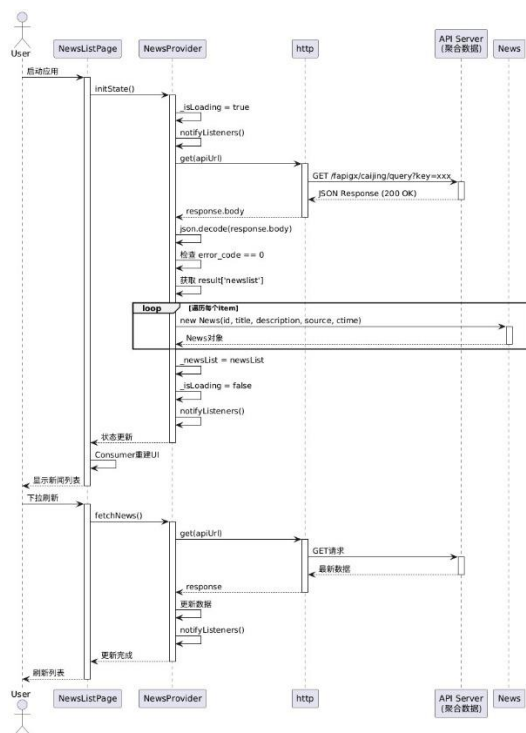
发起网络请求获取 API 数据

解析 JSON 并渲染列表

用户下拉触发刷新 → 重新请求数据

点击新闻项 → 弹出详情弹窗

流程覆盖数据加载、刷新、交互全链路，清晰体现跨平台列表页的标准业务逻辑。



六、实验总结与思考

(一) 实验总结

本次实验完整实现了 4 种跨平台框架的新闻列表模块开发，掌握了 RESTful API 数据请求、JSON 解析、下拉刷新、弹窗交互等核心功能的跨平台实现方式。通过实验，我理解了不同跨平台框架的开发语法、状态管理方式与性能差异，体验了 AI 工具在多框架代码生成中的辅助作用，建立了对跨平台应用开发流程的系统认知。实验中，各框架的核心功能均成功实现，验证了跨平台技术在统一功能需求下的多方案可行性。

(二) 问题与解决

1. 问题 1：Flutter 项目中，ListView.builder 渲染大量数据时出现卡顿。

解决方法：通过优化列表项构建逻辑，减少不必要的组件重建，结合 AutomaticKeepAliveClientMixin 缓存列表项，提升渲染性能。

2. 问题 2：Uniapp 中 uni.request 请求 API 数据时出现跨域问题。

解决方法：通过 HBuilderX 配置代理服务器，或在 API 端配置 CORS 跨域策略，成功解决跨域请求限制。

3. 问题 3：MAUI 项目中，HttpClient 请求数据后，列表无法自动更新。

解决方法：排查发现 ViewModel 中数据更新后未触发 UI 通知，添加 ObservableProperty 属性，实现数据变更与 UI 更新的双向绑定。

（三）课后思考

1. 框架适用场景分析：Flutter 适合对性能要求高、需自定义 UI 的跨平台应用；React Native 适合前端开发者快速开发轻量级应用；Uniapp 适合需快速兼容多端的小程序/APP 项目；MAUI 适合 .NET 生态下的企业级跨平台应用开发。
2. 跨平台框架选型：选型需结合开发团队技术栈、应用性能需求、目标平台与维护成本综合考虑，优先选择与团队现有技术栈匹配、生态完善的框架。
3. 传感器在移动应用中的应用：GPS、加速度计等传感器可在校园场景中实现校园导航、运动数据采集、设备定位等功能，结合跨平台框架的插件生态，可快速拓展应用场景。

七、实验心得

通过本次实验，我对跨平台应用开发有了全面的实践认知，从多框架的语法差异到功能实现，从数据交互到性能对比，逐步掌握了跨平台开发的核心流程。在同时学习 4 种框架的过程中，我不仅提升了多语言、多框架的代码编写能力，也学会了对比分析不同技术方案的优缺点。AI 工具的辅助让我在多框架代码编写中少走了很多弯路，也让我意识到合理利用工具可大幅提升开发效率。本次实验为后续跨平台应用的开发与技术选型打下了坚实基础，也让我对移动应用开发的多样性有了更深刻的理解。